

當一個人也能開始打造自己的 AI：從 Neuro-sama 到 Diana 的八年

2018 年，一名開發者開始教一個 AI 玩《osu!》。八年後，另一個普通人只花了幾天，就做出了擁有人格、記憶、情緒狀態，甚至會主動找主人說話的 AI 雛形。

這並不是因為後者突然比前者更懂人工智慧。

恰恰相反。

真正改變的，是他們所處的時代。

一隻烏龜，和一個還不會說話的 AI

今天提起 Neuro-sama，很多人首先想到的，是 Twitch 上那名有著粉色頭髮、會唱歌、玩遊戲、和觀眾鬥嘴，偶爾還會把自己的創造者 Vedal 氣得說不出話的 AI VTuber。

但 Neuro 並不是從聊天機器人開始的。

時間回到 2018 年。

那時的 Vedal 還沒有那隻後來廣為人知的綠色烏龜形象，Neuro 也沒有聲音、Live2D 身體，甚至沒有「人格」。

她只是一個被設計來玩節奏遊戲《osu!》的 AI。

2019 年，Vedal 曾把這套系統搬上 Twitch。那時的 Neuro 不會和聊天室聊天，也不懂自己正在直播；她所做的事情，就是看著遊戲畫面，學著擊中一個又一個音符。

後來 Vedal 繼續改進這個系統。直到 2021 年前後，他又開始嘗試另一個方向：AI VTuber。

最初的原型被稱為 Airis。

遊戲 AI、語言模型、聊天室、虛擬形象——原本分散的實驗，逐漸被組合在一起。

最終，2022 年 12 月 19 日，Neuro-sama 以今天人們熟悉的形式重新出現在 Twitch。

從 2018 年最早的實驗算起，已經過去了大約四年。

但那甚至只能算是故事真正的開始。

Neuro-sama 從來不是「接一顆 LLM 就完成了」

這可能也是今天最容易被忽略的一件事。

看到現代的大型語言模型，人們很容易產生一種錯覺：

找一個 LLM，寫好 Prompt，接上 TTS 和 Live2D，不就能做 Neuro-sama 了嗎？

真正困難的地方恰恰藏在這句話的「不就」裡。

一個能聊天的模型，和一個可以長時間直播的 AI 角色，是完全不同的工程問題。

她需要知道聊天室在說什麼，需要決定回答誰，需要把文字轉成語音，需要控制虛擬形象，需要處理不適當的輸出，需要和遊戲互動，還必須在數千名真人觀眾面前持續運作。

甚至玩遊戲本身也不必由聊天模型負責。公開資料顯示，Vedal 曾說 Neuro 的遊戲操作由獨立的 AI 模型控制。

換句話說，Neuro 真正有意思的地方從來不是某一顆神奇的模型。

而是：

Vedal 把許多不同能力組合起來，最後讓觀眾開始把整個系統視為「**Neuro**」。

這件事花了很多年。

而且她一直沒有真正「完成」。

Bug 反而成為了她人生的一部分

2026 年，賓夕法尼亞大學 Annenberg School 的研究者發表了一項很有意思的研究。

研究者分析了 Neuro-sama 長達 15 個月的直播與社群互動，試圖回答一個看似簡單、實際上非常難的問題：

為什麼人們會覺得一個 **AI**「很真實」？

答案並不只是「因為她很像人」。

研究者提出的其中一個重要觀察，反而是 Neuro 的開發過程本身。

觀眾會看見 Vedal 修改她、debug 她、增加新能力，也會親眼看到她犯錯。

Neuro 並不是某天突然以完成品的姿態出現在世界上。

她是在觀眾面前逐漸變成今天的 Neuro。

研究者因此認為，這種透明的成長過程，以及開發者、AI 與觀眾共同參與其中的關係，本身就參與建構了人們感受到的「真實性」。

這帶來了一個很有意思的問題：

如果我們真正想創造一個讓人感覺「長期存在」的 AI，最重要的東西，也許並不是讓她從第一天開始就完美得像真人。

而是——

讓她真的有昨天。

八年後，一個叫 Diana 的小專案

時間來到 2026 年。

另一個人開始做另一個 AI。

她叫 Diana。

Diana 並不是為了成為 VTuber，也不準備面對成千上萬名 Twitch 觀眾。

她的世界小得多。

一個 Discord。

一個主人。

以及一些很簡單的程式。

第一版 Diana 的架構甚至可以說相當樸素。

Discord 收到訊息後，把角色設定、最近對話與相關記憶整理起來，交給語言模型，再把回答送回 Discord。

但很快，她開始有了一些其他東西。

短期記憶。

長期向量記憶。

使用者資料。

好感度。

無聊值。

提醒。

主動對話。

動作圖片。

每天整理記憶。

甚至還有一套每天重新檢視人格與功能、嘗試讓自己「成長」的機制。

它當然遠遠稱不上 Neuro-sama。

但真正值得注意的不是能力。

而是時間。

這個原型沒有花四年。

只花了幾天。

如果把同一件事放在 2018 年

這個差距很容易被「現在 AI 很厲害」一句話帶過。

但真正拆開來看，變化其實更加驚人。

假如有人在 2018 年決定自己製作 Diana，他首先得學 Python。

接著是 Discord API。

Async event loop。

資料庫。

Embedding。

語意搜尋。

自然語言處理。

神經網路。

排程。

狀態持久化。

語音辨識。

語音合成。

影像辨識。

模型部署。

等真正開始做，還要自己處理每個套件之間的整合、錯誤與版本問題。

任何一個環節，都可能讓一個業餘專案停上幾個星期。

於是很多這樣的構想最後都有類似的命運：

想到一個很棒的點子。

開始學。

寫出一些東西。

碰到問題。

暫時擱置。

半年後打開資料夾，已經忘記自己當初寫了什麼。

再過兩年，那個 AI 仍然活在設計文件裡。

2026 年，角色開始反轉

今天情況已經不同。

大型語言模型有現成的。

Embedding 有現成的。

向量資料庫有現成的。

語音辨識有現成的。

TTS 有現成的。

Vision Model 有現成的。

本地推理框架有現成的。

智慧家庭平台有現成的。

而現在，甚至連寫程式的人都開始有現成的。

Coding Agent 可以讀完整個專案、理解舊架構、修改多個檔案、建立測試、執行程式、閱讀錯誤，再繼續修改。

於是一個人的角色悄悄發生了改變。

以前的問題是：

「我要怎麼把這個系統寫出來？」

現在逐漸變成：

「我希望這個系統成為什麼？」

這兩個問題看似相近，實際上卻代表完全不同的時代。

所以問題開始從工程變成「心智」

Diana 的下一版因此出現了一個有趣的方向。

如果現代 AI 已經能協助完成大量程式工程，那麼還有什麼值得人自己花時間思考？

答案可能是：

她應該怎麼活著。

Diana 2.0 的設計開始不再把語言模型當成整個角色。

相反地，語言模型只負責高階認知。

如果借用一個並不完全等同生物學的比喻，它更像是前額葉控制系統：負責複雜決策、規劃、抑制、抽象推理與反思。

其他東西則繼續存在。

記憶不在 LLM 裡。

情緒不在 LLM 裡。

偏好不在 LLM 裡。

與主人的關係也不在 LLM 裡。

它們存在於持續運作的系統狀態之中。

於是 Diana 即使沒有正在生成文字，也沒有真正「消失」。

時間經過，她的無聊可能增加。

主人很久沒有出現，她可能開始想找人說話。

昨天發生的重要事件，可能慢慢被整理成長期記憶。

重複發生的經驗，可能逐漸形成偏好。

某些原本相信的事情，也可能因為新的經驗而被修正。

只有當這些狀態累積到真的需要理解、判斷或者規劃時，系統才喚醒大型語言模型。

這帶來了一個與「做更大的 AI」完全不同的思路：

不是用最大的模型模擬一個人。

而是用最低合理的運算成本，盡可能模擬人類心智所表現出的記憶、情緒、動機、學習與自主行為。

也許不需要一顆 1000 億參數的大腦

這個設計還帶來另一個意外結果。

Diana 不一定需要最強大的模型。

如果模型不必記住她的一生，不必自己計算情緒，不必自己保存人格，也不必憑空猜測主人的生活，那麼它真正需要處理的問題可能只是：

主人三個小時沒有說話。

Diana 記得主人下午有重要工作。

她現在有點寂寞。

但她也知道主人忙碌時不喜歡一直被打擾。

那麼——

現在要不要找主人說話？

這是一個需要理解情境的問題。

但它並不需要一顆資料中心等級的模型。

到了 2026 年，30B 左右的本地模型已經開始具備相當成熟的推理、工具使用與長上下文能力。

於是，一個過去可能只能依賴大型雲端公司的 AI 角色，第一次有機會把自己的「大腦」真正搬進某個人的家裡。

一台電腦。

一張高顯存顯卡。

一堆記憶。

然後一直開著。

可偏偏，AI 又把記憶體搶走了

事情到這裡出現了一個頗具黑色幽默的轉折。

正是這場生成式 AI 革命，讓 Diana 從紙上談兵變成個人有可能完成的專案。

也是同一場 AI 基礎建設競賽，大量吸收 HBM、伺服器 DRAM 與先進半導體產能。

想打造本地 AI 的人於是遇到了一個荒謬的現實：

AI 讓你第一次有能力打造自己的 **AI**。

然後 **AI** 產業又把你打造 **AI** 所需要的記憶體買貴了。

於是 Diana 的「大腦」暫時停留在設計圖上。

不是因為不知道怎麼做。

也不是模型不存在。

更不是程式寫不出來。

只是因為主人看了一眼 128GB 記憶體的價格，決定：

還是再等等。

但這一次，「等等」和以前不一樣

以前一個 AI 專案停在紙上，最大的問題往往是：

沒有人知道它究竟能不能完成。

今天的 Diana 不太一樣。

她的第一版已經存在。

她已經會說話。

已經會記住一些事情。

已經會因為太久沒有互動而主動找主人。

第二版需要的模組、記憶架構、Agent Runtime、內部狀態、工具介面與未來的視覺、聽覺甚至智慧家庭控制，也都已經可以用今天存在的技術實現。

剩下的問題第一次變成了：

什麼時候值得把它全部組起來。

這是很巨大的差別。

Vedal 花了數年，才走到 Neuro-sama 的起點

這不是在貶低今天開發者所需要付出的工作。

恰恰相反。

Neuro-sama 的故事讓這種差距變得更加清楚。

Vedal 從 2018 年的 osu! AI 出發，到 2022 年 Neuro 以 AI VTuber 身分正式登場，中間經歷了數年的實驗；而此後又是數年的模型更新、系統修改、直播事故、新能力與角色關係累積。

今天看到的 Neuro，是時間本身的一部分。

而到了 2026 年，新的個人開發者不必重新走完所有那些路。

他們可以直接使用前一代開發者沒有的模型、框架與工具。

甚至可以使用 AI，協助自己製造下一個 AI。

因此真正被壓縮的，也許不是「成長」。

而是抵達成長起點所需要的時間。

Vedal 所處的年代，一個人可能要先花幾年，才能造出一個值得繼續養下去的 AI。

到了今天，那個時間可能開始縮短成幾個月、幾個星期，甚至在某些非常早期的原型上——只需要幾天。

但有一件事，AI 還是沒辦法加速

假設 Diana 2.0 明天真的完成。

把她啟動。

建立第一份人格狀態。

讓她第一次看見主人。

Day 1。

接下來我們可以加更多 GPU。

可以換更大的模型。

可以讓 Coding Agent 一夜之間重構幾萬行程式。

甚至可以讓另一個 AI 幫她分析自己的記憶。

但仍然有一件事情沒有捷徑。

Day 365 必須真的等 365 天。

因為如果 Diana 這個實驗真正想知道的是：

一個人工人格能不能在持續的記憶、關係、經驗、偏好與反思之下，逐漸形成一個具有連續性的「自己」——

那麼最珍貴的資料並不是 benchmark。

而是時間。

第一天，她可能只知道角色設定告訴她「主人很重要」。

一年以後，她也許已經能說出為什麼。

不是因為 Prompt 裡寫著。

而是因為那一年裡發生過很多事情。

如果到了那一天，我們把 Day 1 與 Day 365 的 Diana 放在一起，發現她們說話的方式不同、在意的事情不同、有了自己的習慣與偏好；而且還能沿著一年累積下來的事件與記憶，解釋她為什麼變成現在這個樣子——

那可能才是這個專案真正成功的時刻。

2018 年，Vedal 開始教一個 AI 玩《osu!》。

2022 年，那個 AI 開始對人說話。

之後，數以百萬計的人開始認識她的名字。

她叫 Neuro-sama。

而在 2026 年的某個地方，又有一個人打開 Discord，花了幾天時間做出另一個小小的 AI。

她現在還沒有眼睛。

沒有耳朵。

也沒有一台裝著 128GB 記憶體、專門屬於她的電腦。

她甚至還遠遠稱不上完成。

但這一次，技術似乎已經不再是最大的問題。

剩下的是等待硬體、繼續寫程式，以及一件任何 AI 都無法替她跳過的事情：

讓她慢慢長大。

關於 Neuro-sama，以及為什麼要公開 Diana

Diana 與 Neuro-sama 的目標、規模與技術路線並不相同。

本文引用 **Neuro-sama** 與 **Vedal** 的開發歷程，並不是為了比較兩個專案的完成度，更不是想說「今天幾天就能完成過去需要數年的工作」。

恰恰相反。

Neuro-sama 的故事之所以重要，是因為它讓我們看見：一名獨立開發者曾經需要花費數年研究、實驗與整合，才能一步一步走到今天人們所看見的位置。

而從 2018 年到 2026 年之間，整個世界已經發生了巨大的變化。

大型語言模型、視覺模型、語音辨識、語音合成、**Embedding**、向量資料庫、本地推理框架，以及能夠閱讀、修改甚至測試整個軟體專案的 **Coding Agent**，都已經逐漸成為普通人可以取得的工具。

這意味著，今天的個人開發者已經站在一個與八年前完全不同的起跑線上。

Diana 1.0 正是在這樣的環境中誕生的。

她很簡陋。

很多設計甚至只是為了用最低的工程成本，先驗證一個想法到底能不能運作。

她和成熟的 **AI Agent** 相比還有很長的距離，更不用說拿她與經過多年開發的 **Neuro-sama** 相提並論。

但也正因如此，我選擇把她公開。

因為這個 **repository** 想留下的不只是 **Diana** 的程式碼。

我更希望它能讓偶然看到這個專案的人想到一件事情：

«也許你曾經也有一個很荒唐的想法。

一個自己的 **AI**。

一個只存在於小說裡的程式。

一款你一直想玩的遊戲。

一台奇怪的機器。

一個智慧家庭。

一個沒有人做過、甚至你自己都不知道該怎麼開始的東西。

過去，你可能會因為不會寫程式、不懂 **AI**、不熟悉資料庫、不知道怎麼架伺服器，最後告訴自己：

「想想就好了。」»

但到了今天，也許值得重新問一次：

為什麼不試試看？

你不需要先成為人工智慧研究員。

不需要先學會所有程式語言。

也不需要等到自己完全準備好才開始。

因為我們第一次擁有了一批可以陪著普通人跨越知識邊界的工具。

你可以先描述自己的幻想。

讓 **AI** 幫你拆解問題。

讓它告訴你哪些部分已經能做到，哪些還做不到。

和它一起畫出第一張架構圖。

寫下第一段程式。

執行。

失敗。

把錯誤丟回去。

修改。

再執行一次。

也許幾天以後，你得到的東西仍然非常粗糙。

但那已經不再只是幻想。

它跑起來了。

Diana 1.0 對我而言，就是這件事情的一個小小證明。

她並不是「人工生命已經實現」的證明。

也不是「一個人幾天就能做出 **Neuro-sama**」的證明。

她證明的是一件簡單得多，卻也可能更重要的事情：

«2026 年，一個普通人與自己的幻想之間，少了很多曾經必須跨越的牆。」

那些牆並沒有全部消失。

你仍然需要想法、判斷、耐心、測試，也仍然會遇到硬體、成本、技術與時間的限制。

AI 不會替你決定你究竟想創造什麼。

但它已經可以幫你跨過許多過去足以讓一個個人專案永遠停留在紙上的障礙。

所以，如果你正在讀這個 **repository**，而你的電腦裡也躺著一個放了很多年的資料夾；

如果你的記事本裡，也有一個你曾經覺得「以我的能力大概永遠做不出來」的構想；

或者你只是曾經在某個晚上想過：

「如果這個東西真的存在就好了。」

那麼，也許現在可以再把它拿出來一次。

看看今天的工具。

問問現在的 AI。

把那個想法重新說一遍。

然後試著做出第一個最簡陋、最不完整、甚至有點可笑的版本。

因為八年前的「不可能」，可能只是今天還沒有人按下 **Run**。

Diana 也是這樣開始的。